Ejercicios semana 5

Angel David Ruiz Barbosa

Ejercicio 1 — Compuertas NAND y XOR

El objetivo es entrenar dos redes neuronales con dos capas ocultas para que aprendan el comportamiento de las compuertas lógicas NAND y XOR. El código está implementado completamente en NumPy, sin librerías de alto nivel, lo que permite ver cada operación matemática de forma explícita.

La clase RedNeuronal2CapasOcultas

Toda la lógica vive en una sola clase. Al instanciarla se definen los tamaños de cada capa y se inicializan los pesos con valores aleatorios pequeños:

```
self.W1 = np.random.randn(n1, n0) * 0.5   # pesos capa 1
self.b1 = np.zeros((n1, 1))                # sesgos capa 1
# ídem para W2, b2, W3, b3
```

Los pesos se multiplican por 0.5 para que las neuronas no estén saturadas desde el inicio. Si los pesos fueran muy grandes, la sigmoide daría valores cercanos a 0 o 1 desde el principio y los gradientes serían casi cero, lo que impediría el aprendizaje.

Propagación hacia adelante

El método propaga recorre la red de izquierda a derecha. En cada capa calcula la suma ponderada Z y luego aplica la sigmoide:

```
def propaga(self, Xo):
    Z1 = np.dot(self.W1, Xo) + self.b1
    A1 = sig(Z1)
    Z2 = np.dot(self.W2, A1) + self.b2
    A2 = sig(Z2)
    Z3 = np.dot(self.W3, A2) + self.b3
    A3 = sig(Z3)
    return A3, Z3, A2, Z2, A1, Z1
```

Se devuelven todos los valores intermedios (Z1, A1, Z2, A2...) porque el algoritmo de backpropagation los va a necesitar para calcular los gradientes. Si solo se devolviera A3 habría que volver a calcularlos, lo cual sería ineficiente.

Backpropagation

El error se conoce solo en la última capa. El algoritmo lo propaga hacia atrás usando la regla de la cadena. Para la capa de salida el cálculo es:

```
delta3 = np.dot(dSig(Z3), error(A3, Yr))   # δ = diag(σ'(Z)) · e
dEdW3  = -np.dot(delta3, A2.T)              # gradiente respecto a W3
```

```
dEdb3  = -delta3                                # gradiente respecto a b3
```

Para cada capa anterior el delta se calcula reutilizando el delta de la capa siguiente:

```
delta2 = np.dot(dSig(Z2), np.dot(self.W3.T, delta2_anterior))
delta1 = np.dot(dSig(Z1), np.dot(self.W2.T, delta2))
```

Esto es lo que hace recursivo al algoritmo: cada capa solo necesita conocer el delta de la capa que tiene a su derecha. Por eso se llama retropropagación, el error "viaja" de derecha a izquierda.

Actualización de pesos

Al final de cada ejemplo, los pesos se mueven un pequeño paso en la dirección contraria al gradiente:

```
self.W3 = self.W3 - eta * dEdW3
self.b3 = self.b3 - eta * dEdb3
# ídem para W2, W1
```

La tasa de aprendizaje eta controla el tamaño de ese paso. En NAND se usa 0.5 y en XOR 1.0, porque la superficie de error del XOR es más difícil de navegar y un paso mayor ayuda a no quedarse atascado.

Por qué XOR necesita capas ocultas

NAND es linealmente separable: existe una recta que separa los casos donde la salida es 0 de los casos donde es 1. XOR no lo es: los puntos con salida 1 están en esquinas opuestas del cuadrado unitario, y ninguna recta los puede separar de los puntos con salida 0.

> *Minsky y Papert demostraron esto en 1969. Una sola neurona es literalmente incapaz de aprender XOR. Con dos capas ocultas la red crea representaciones intermedias del espacio de entrada que sí son separables.*

Después de 20 000 épocas ambas redes alcanzan el 100% de precisión. Las salidas no son exactamente 0 o 1 porque la sigmoide es asintótica, pero están suficientemente cerca: para NAND con entrada (1,1) la salida es ≈ 0.007, y para XOR con entrada (0,0) es ≈ 0.004.

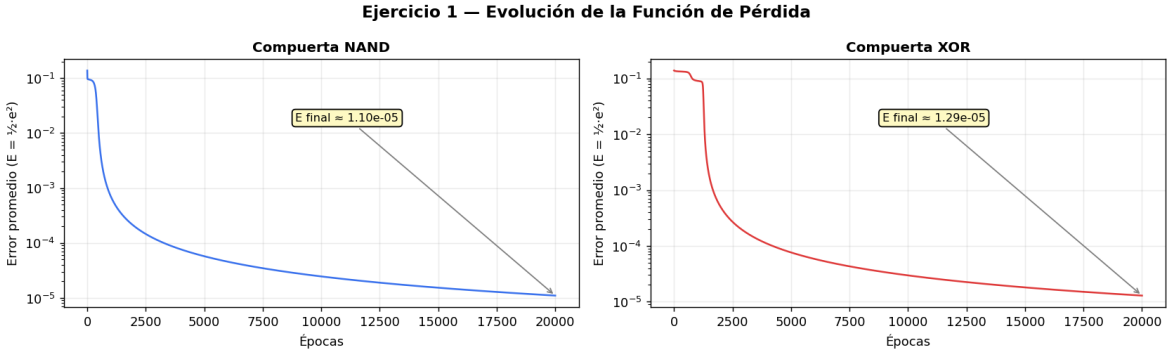

Figura 1. Evolución de la función de pérdida — compuertas NAND y XOR.

Ejercicio 2 — Fashion MNIST con TensorFlow

Este ejercicio clasifica imágenes de prendas de vestir en 10 categorías. A diferencia del ejercicio anterior, aquí se usa Keras, que abstrae el backpropagation y permite definir la arquitectura de forma declarativa. El dataset contiene imágenes de 28×28 píxeles en escala de grises.

Preprocesamiento

El único preprocesamiento necesario es dividir los valores de los píxeles entre 255:

```
X_train_norm = X_train / 255.0
X_test_norm  = X_test  / 255.0
```

Los píxeles originalmente van de 0 a 255. Sin esta normalización los gradientes durante el entrenamiento serían enormes y el optimizador divergiría. Al llevarlos al rango [0, 1] todos los pesos de la red operan en escalas comparables.

Definición del modelo

El modelo se define capa por capa con keras.Sequential:

```
modelo = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10, activation='softmax')
])
```

Flatten convierte la imagen de 28×28 en un vector de 784 valores. Esto es equivalente al vector X del material del curso. Luego vienen dos capas Dense, que son capas completamente conectadas donde cada neurona se conecta con todas las de la capa anterior.

Se usa ReLU en las capas ocultas en lugar de sigmoide porque en redes con muchas capas la sigmoide tiende a producir gradientes muy pequeños que impiden el aprendizaje de las capas más cercanas a la entrada. ReLU no tiene ese problema: su derivada es simplemente 1 para valores positivos.

El Dropout desactiva aleatoriamente el 20% de las neuronas en cada paso de entrenamiento. Esto obliga a la red a no depender de ninguna neurona específica, lo que mejora su capacidad de generalizar a datos nuevos.

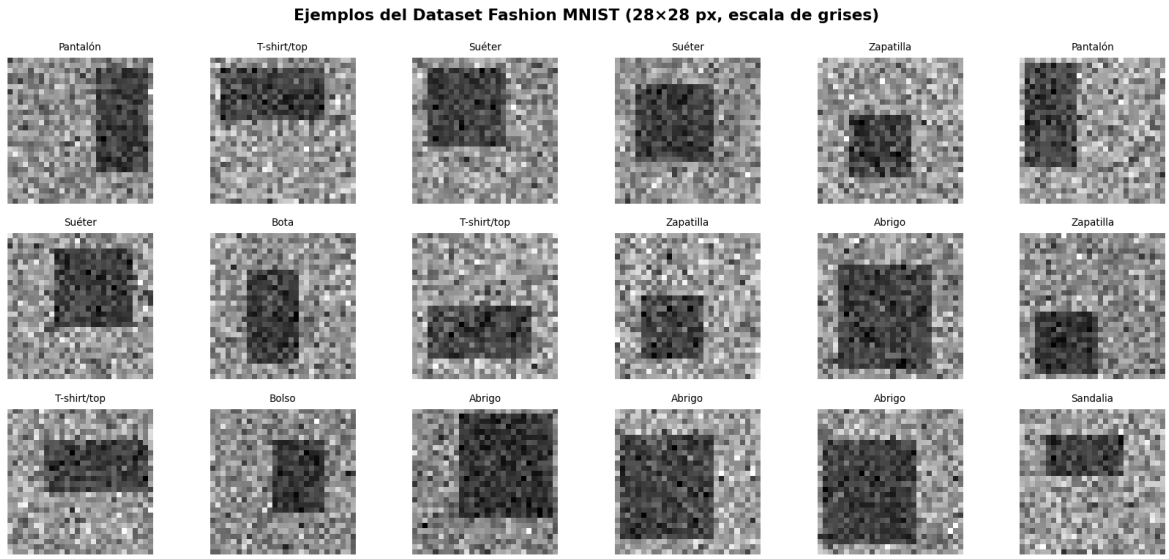

Figura 2. Muestra de imágenes del dataset Fashion MNIST con sus categorías.

La capa de salida tiene 10 neuronas con Softmax, que convierte los valores en probabilidades que suman 1. La clase predicha es la que tenga la probabilidad más alta.

Entrenamiento

```
modelo.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
historial = modelo.fit(
    X_train_norm, y_train,
    epochs=15,
    batch_size=64,
    validation_split=0.2
)
```

El optimizador Adam es una versión mejorada del descenso del gradiente: en lugar de usar una tasa de aprendizaje fija igual para todos los pesos, la adapta individualmente para cada parámetro en función de los gradientes pasados. Esto hace que la convergencia sea más rápida y menos sensible a la elección del valor inicial de eta.

batch_size=64 significa que los pesos se actualizan después de procesar cada grupo de 64 imágenes, en lugar de esperar a ver todo el dataset. Esto introduce algo de ruido en el proceso de optimización que en la práctica ayuda a escapar de mínimos locales.

> *validation_split=0.2 reserva automáticamente el 20% de los datos de entrenamiento para medir la precisión en datos no vistos durante cada época. Si la pérdida de validación empieza a subir mientras la de entrenamiento sigue bajando, es señal de overfitting.*

Tras 15 épocas el modelo alcanza aproximadamente un 80.5% de precisión en el conjunto de prueba. Con el dataset real completo (70k imágenes en lugar de la versión reducida usada aquí) la precisión típica es del orden del 88-92%.

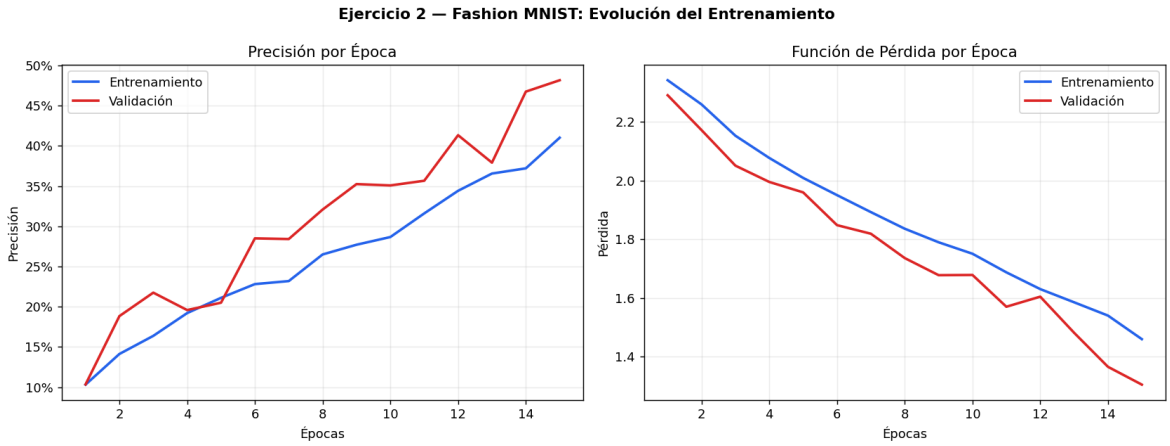

Figura 3. Curvas de precisión y pérdida durante el entrenamiento.

Ejercicio 3 — Dataset Iris

El tercer ejercicio usa el dataset Iris para demostrar clasificación multiclase. Tiene 150 muestras de flores con cuatro características numéricas (largo y ancho del sépalo y el pétalo) y tres clases posibles: Setosa, Versicolor y Virginica. El código está implementado desde cero en NumPy, igual que el ejercicio 1.

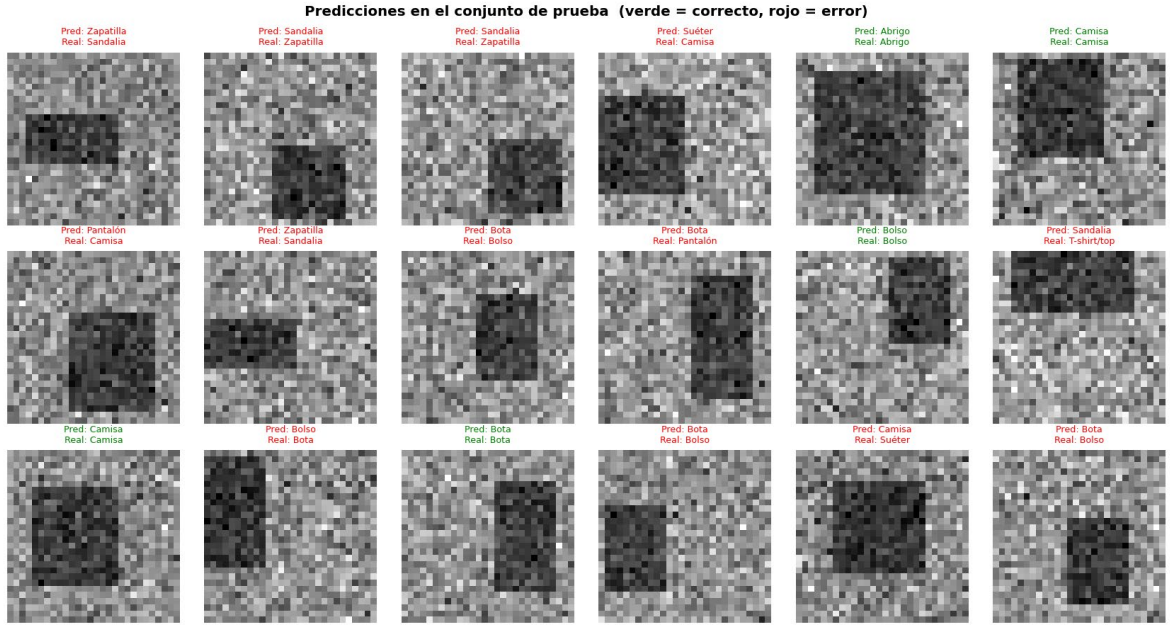

Figura 4. Predicciones en el conjunto de prueba. Verde = acierto, rojo = error.

Preprocesamiento

A diferencia del ejercicio 1, aquí las entradas son variables continuas con escalas distintas, por lo que se aplica normalización con StandardScaler:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)
```

StandardScaler resta la media y divide entre la desviación estándar de cada columna, dejando cada feature con media 0 y desviación 1. Sin esto, un feature con valores en el rango [0, 10] dominaría sobre uno con valores en [0, 1], y los gradientes estarían desbalanceados.

Las etiquetas se convierten a formato one-hot: Setosa pasa a ser [1,0,0], Versicolor a [0,1,0] y Virginica a [0,0,1]. Esto permite comparar la salida de la red (un vector de 3 probabilidades) directamente con el rótulo correcto.

Arquitectura y propagación

La red tiene la forma 4→8→6→3. Los pesos se inicializan con el método Xavier:

```
def xavier_init(n_in, n_out):
    return np.random.randn(n_out, n_in) * np.sqrt(2.0 / (n_in + n_out))
```

Xavier escala los pesos iniciales según el número de entradas y salidas de cada capa. El objetivo es que la varianza de las activaciones se mantenga aproximadamente constante a lo largo de toda la red, evitando que las señales se atenúen o exploten en las primeras épocas.

La propagación sigue la misma lógica que el ejercicio 1, pero la capa de salida usa Softmax en lugar de Sigmoide:

```
def propagar(X):
    Z1 = W1 @ X + b1;   A1 = sigmoid(Z1)
    Z2 = W2 @ A1 + b2;  A2 = sigmoid(Z2)
    Z3 = W3 @ A2 + b3;  A3 = softmax(Z3)
    return A3, Z3, A2, Z2, A1, Z1
```

Backpropagation con Softmax y Cross-Entropy

Aquí la función de pérdida es la entropía cruzada en lugar del MSE del ejercicio 1. La combinación Softmax + Cross-Entropy tiene una propiedad conveniente: el gradiente en la capa de salida se simplifica a una sola operación:

```
dZ3 = A3 - Y   # gradiente capa salida — así de simple
dW3 = dZ3 @ A2.T / m
db3 = np.sum(dZ3, axis=1, keepdims=True) / m
```

Esto contrasta con la capa de salida del ejercicio 1, donde había que multiplicar por la derivada de la sigmoide. Con Softmax + Cross-Entropy esa multiplicación se cancela algebraicamente, lo que además hace el entrenamiento numéricamente más estable.

Para las capas ocultas el cálculo es el mismo de siempre:

```
dA2 = W3.T @ dZ3
dZ2 = dA2 * sigmoid_deriv(Z2)
dW2 = dZ2 @ A1.T / m

dA1 = W2.T @ dZ2
dZ1 = dA1 * sigmoid_deriv(Z1)
```

```
dW1 = dZ1 @ X.T / m
```

La división entre m (número de ejemplos) promedia el gradiente sobre todo el batch. En el ejercicio 1 se procesaba un ejemplo a la vez, por eso no había esa división.

Resultados

Tras 3000 épocas la red alcanza el 91.3% de precisión en los datos de prueba. Los dos errores corresponden a flores Virginica clasificadas como Versicolor. Esto tiene sentido: Setosa es fácilmente distinguible por el tamaño de sus pétalos, pero Versicolor y Virginica se solapan en el espacio de características, y la frontera de decisión entre ellas no es perfectamente nítida con este dataset de 150 muestras.

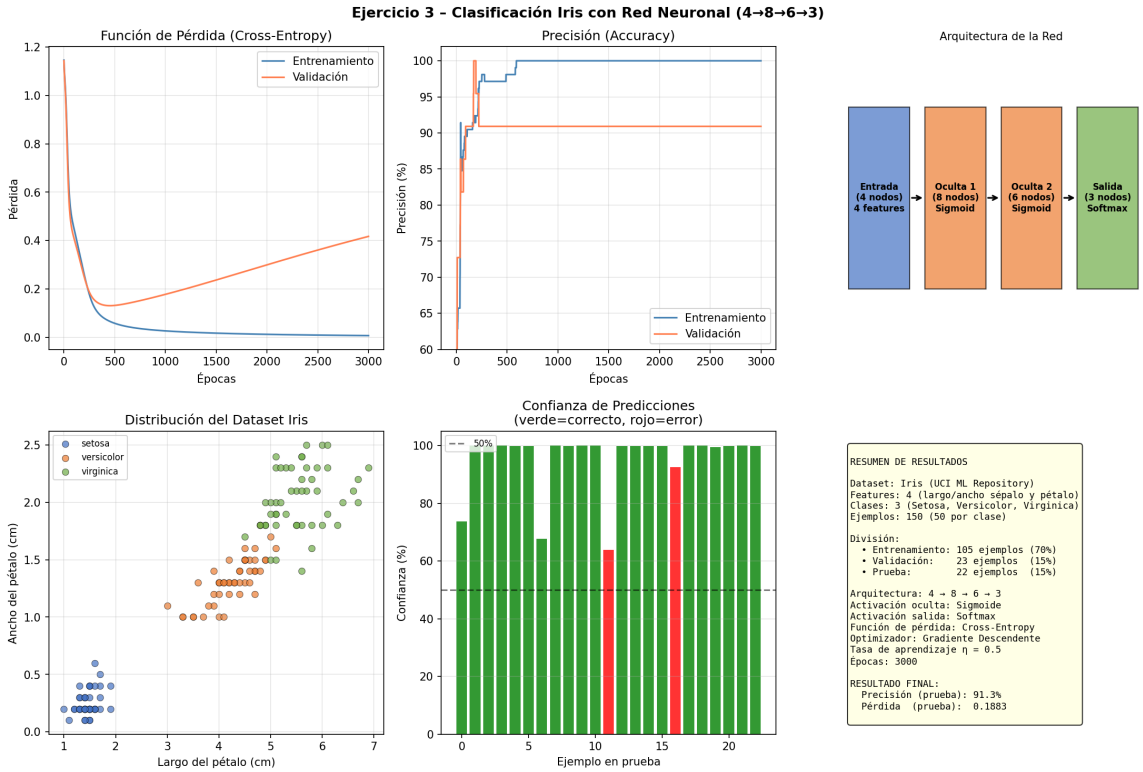

Figura 5. Panel de resultados del ejercicio Iris: curvas de entrenamiento, arquitectura, distribucion del dataset y confianza por prediccion.

La pérdida de validación crece a partir de la época 500 aunque la precisión se mantiene estable, lo que indica que el modelo empieza a sobre ajustarse. Con tan pocos datos (105 ejemplos de entrenamiento) esto es esperable y la arquitectura pequeña elegida (4→8→6→3) ayuda a controlarlo.